

Module 6

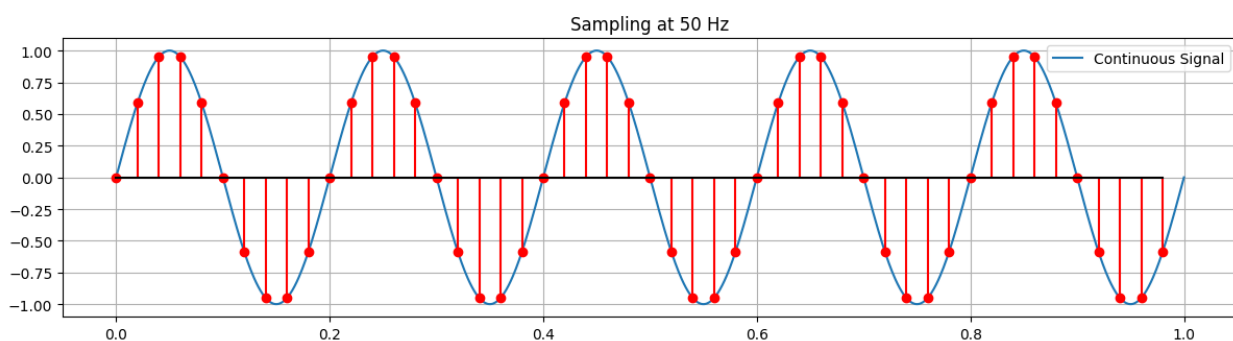
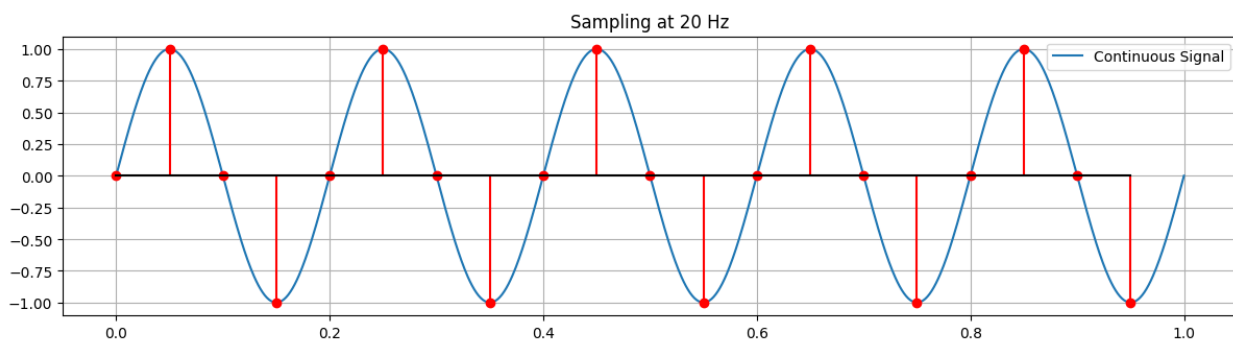
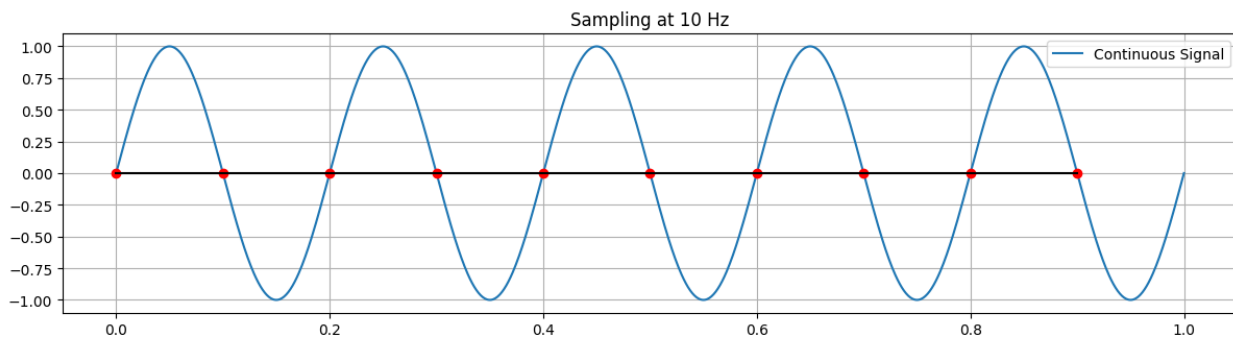
Hints and Solutions

Section A — Basic Problems

Problem 1 Hint

Generate a 5 Hz sine wave. Sample it at 10 Hz, 20 Hz, and 50 Hz. Compare the sampled points with the original signal.

```
import numpy as np
import matplotlib.pyplot as plt
# Continuous signal
f = 5
t = np.linspace(0,1,1000)
x = np.sin(2*np.pi*f*t)
sampling_rates = [10,20,50]
plt.figure(figsize=(12,10))
for i,fs in enumerate(sampling_rates):
    ts = np.arange(0,1,1/fs)
    xs = np.sin(2*np.pi*f*ts)
    plt.subplot(3,1,i+1)
    plt.plot(t,x,label="Continuous Signal")
    plt.stem(ts,xs,linewidth='r-',markerfmt='ro',basefmt='k-')
    plt.title(f"Sampling at {fs} Hz")
    plt.grid(True)
    plt.legend()
plt.tight_layout()
plt.show()
```



Problem 2 Hint

Generate a sinusoidal signal and compare sampling frequencies below, equal to, and above twice the signal frequency.

Problem 3 Hint

Generate one signal and observe the effect of different sampling frequencies.

Theory Used

Under-sampling: $f_s < 2f_m$

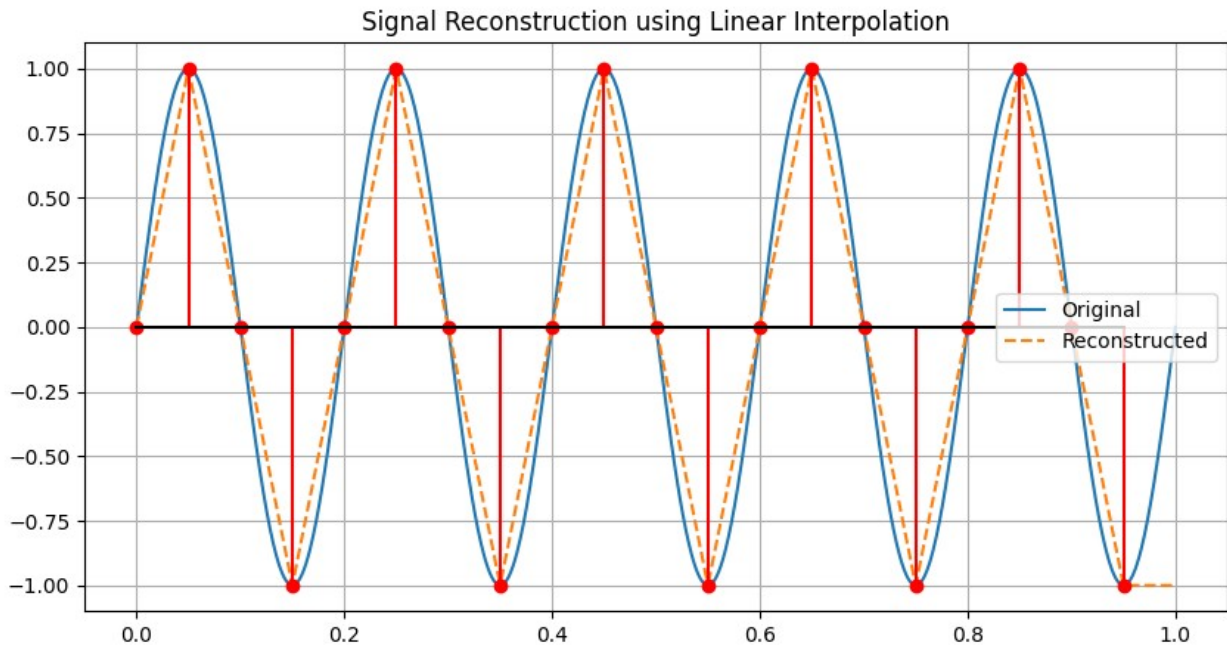
Critical sampling: $f_s = 2f_m$

Over-sampling: $f_s > 2f_m$

Problem 4 Hint

Sample a signal. Reconstruct it using linear interpolation. Compare with the original signal.

```
import numpy as np
import matplotlib.pyplot as plt
f = 5
t = np.linspace(0,1,1000)
x = np.sin(2*np.pi*f*t)
fs = 20
ts = np.arange(0,1,1/fs)
xs = np.sin(2*np.pi*f*ts)
x_interp = np.interp(t,ts,xs)
plt.figure(figsize=(10,5))
plt.plot(t,x,label="Original")
plt.plot(t,x_interp,'--',label="Reconstructed")
plt.stem(ts,xs,linewidth='r-',markerfmt='ro',basefmt='k-')
plt.legend()
plt.grid(True)
plt.title("Signal Reconstruction using Linear Interpolation")
plt.show()
```



Problem 5 Hint

Keep the signal frequency constant and vary the sampling interval.

Section B – Sampling Concepts

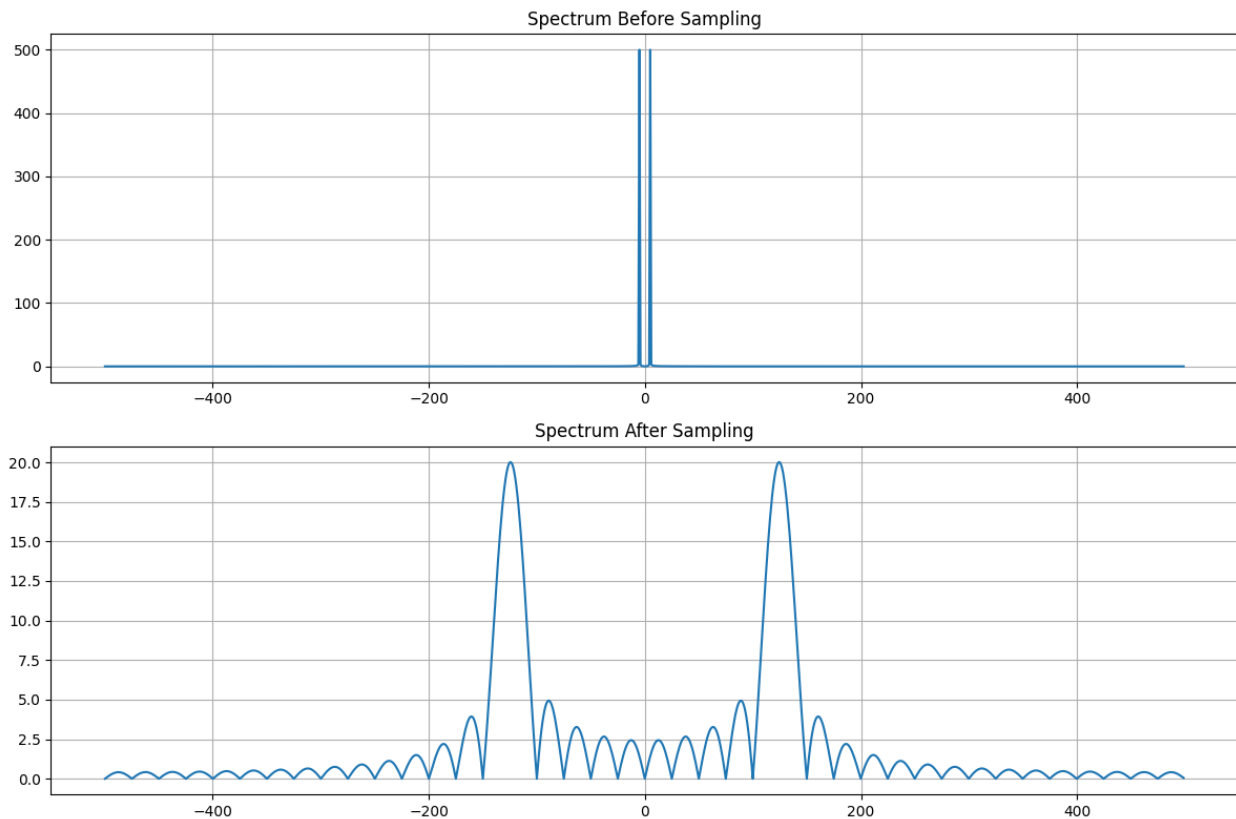
Problem 6 to 9 are theoretical questions. Refer Theory notes for that.

Problem 10 Hint

Compute the FFT of the original signal and the sampled signal. Compare the spectra.

```
import numpy as np
import matplotlib.pyplot as plt
f = 5
fs = 40
t = np.linspace(0,1,1000)
x = np.sin(2*np.pi*f*t)
ts = np.arange(0,1,1/fs)
xs = np.sin(2*np.pi*f*ts)
X = np.fft.fftshift(np.fft.fft(x))
Xs = np.fft.fftshift(np.fft.fft(xs,1000))
freq = np.fft.fftshift(np.fft.fftfreq(1000,t[1]-t[0]))
plt.figure(figsize=(12,8))
plt.subplot(2,1,1)
plt.plot(freq,np.abs(X))
plt.title("Spectrum Before Sampling")
plt.grid(True)
plt.subplot(2,1,2)
```

```
plt.plot(freq,np.abs(Xs))
plt.title("Spectrum After Sampling")
plt.grid(True)
plt.tight_layout()
plt.show()
```

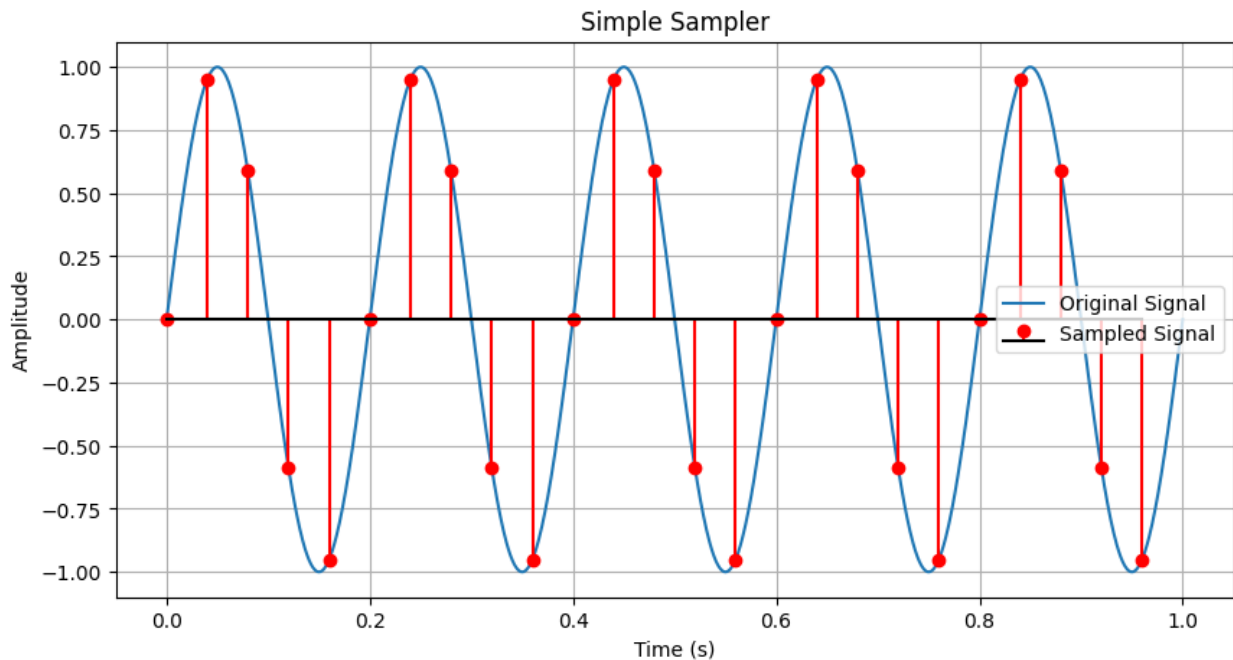


Section C – Programming Problems

Problem 11

```
import numpy as np
import matplotlib.pyplot as plt
# Continuous signal
f = 5
t = np.linspace(0, 1, 1000)
x = np.sin(2*np.pi*f*t)
# Sampling
fs = 25
ts = np.arange(0, 1, 1/fs)
xs = np.sin(2*np.pi*f*ts)
plt.figure(figsize=(10,5))
plt.plot(t, x, label="Original Signal")
plt.stem(ts, xs, linefmt='r-', markerfmt='ro', basefmt='k-',
label="Sampled Signal")
```

```
plt.title("Simple Sampler")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.legend()
plt.grid(True)
plt.show()
```

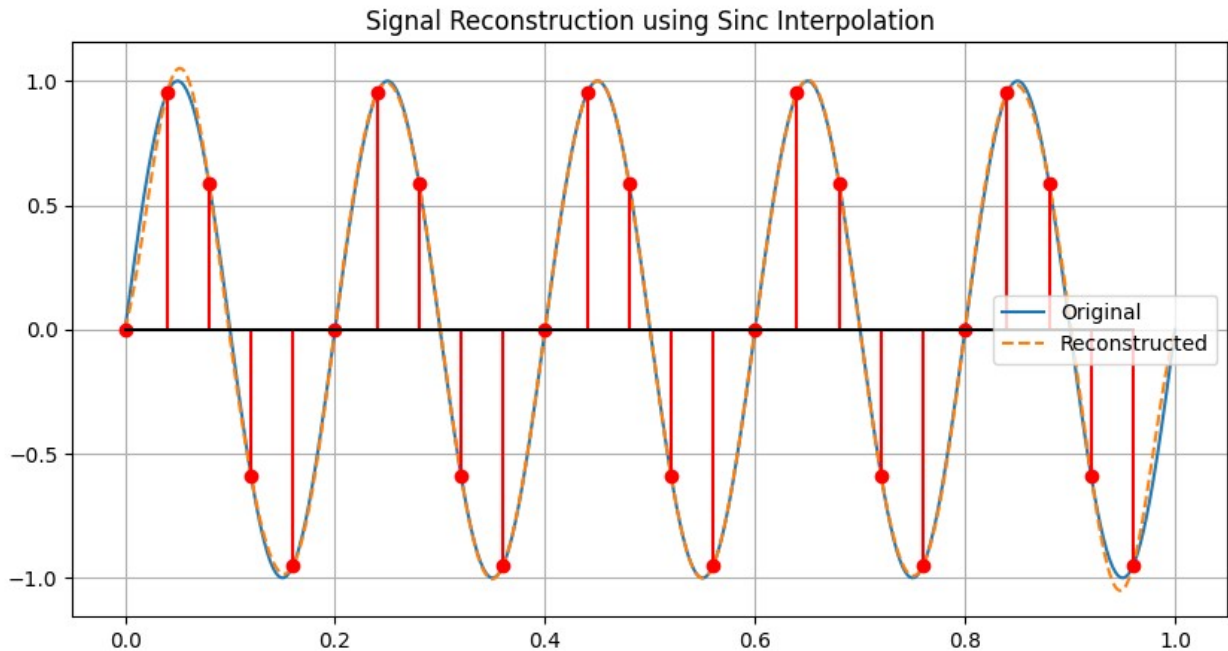


Problem 12 Hint

Sample the signal. Reconstruct it using sinc interpolation. Compare the reconstructed signal with the original.

```
import numpy as np
import matplotlib.pyplot as plt
f = 5
fs = 25
t = np.linspace(0,1,1000)
x = np.sin(2*np.pi*f*t)
ts = np.arange(0,1,1/fs)
xs = np.sin(2*np.pi*f*ts)
# Sinc interpolation
x_rec = np.zeros_like(t)
for i in range(len(ts)):
    x_rec += xs[i]*np.sinc((t-ts[i])*fs)
plt.figure(figsize=(10,5))
plt.plot(t,x,label="Original")
plt.plot(t,x_rec,'--',label="Reconstructed")
plt.stem(ts,xs,linewidth='r-',markerfmt='ro',basefmt='k-')
plt.legend()
```

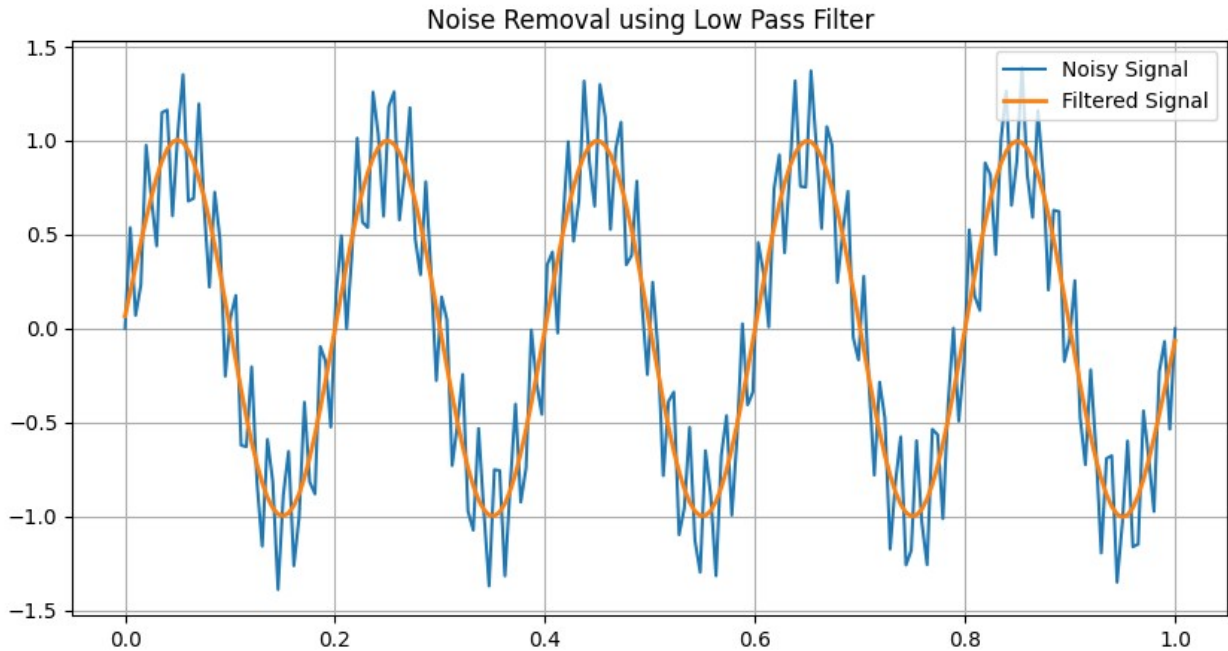
```
plt.title("Signal Reconstruction using Sinc Interpolation")
plt.grid(True)
plt.show()
```



Problem 13 Hint

Add high-frequency noise to a signal. Transform it to the frequency domain. Apply a low-pass filter. Reconstruct the filtered signal.

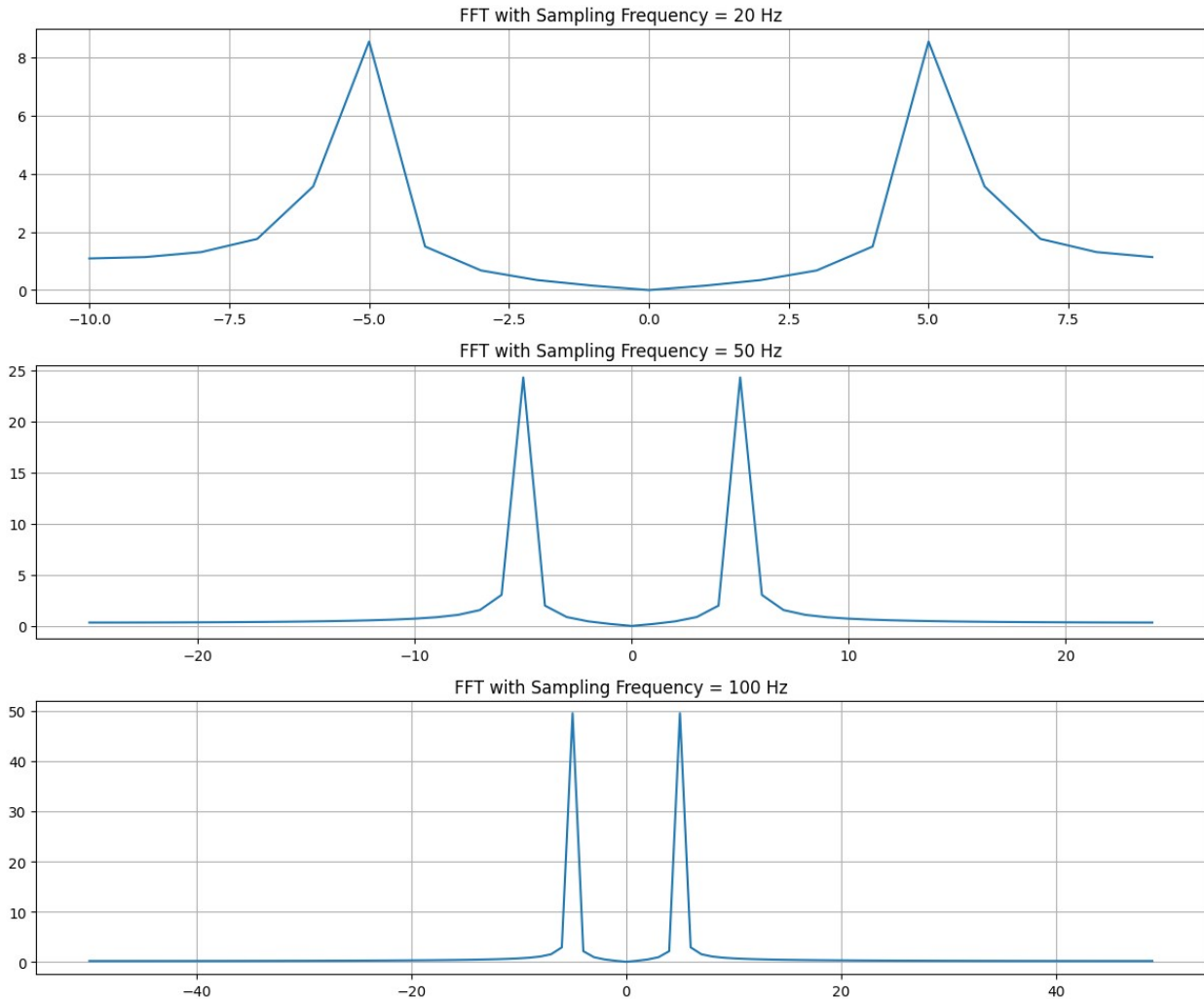
```
import numpy as np
import matplotlib.pyplot as plt
fs = 200
t = np.linspace(0,1,fs)
signal = np.sin(2*np.pi*5*t)
noise = 0.4*np.sin(2*np.pi*60*t)
x = signal + noise
X = np.fft.fft(x)
freq = np.fft.fftfreq(len(X),1/fs)
X[np.abs(freq)>20] = 0
filtered = np.real(np.fft.ifft(X))
plt.figure(figsize=(10,5))
plt.plot(t,x,label="Noisy Signal")
plt.plot(t,filtered,label="Filtered Signal",linewidth=2)
plt.legend()
plt.grid(True)
plt.title("Noise Removal using Low Pass Filter")
plt.show()
```



Problem 14 Hint

Compute the FFT of the same signal using different sampling frequencies.

```
import numpy as np
import matplotlib.pyplot as plt
sampling_rates = [20,50,100]
plt.figure(figsize=(12,10))
for i,fs in enumerate(sampling_rates):
    t = np.linspace(0,1,fs)
    x = np.sin(2*np.pi*5*t)
    X = np.fft.fftshift(np.fft.fft(x))
    freq = np.fft.fftshift(np.fft.fftfreq(len(X),1/fs))
    plt.subplot(3,1,i+1)
    plt.plot(freq,np.abs(X))
    plt.title(f"FFT with Sampling Frequency = {fs} Hz")
    plt.grid(True)
plt.tight_layout()
plt.show()
```



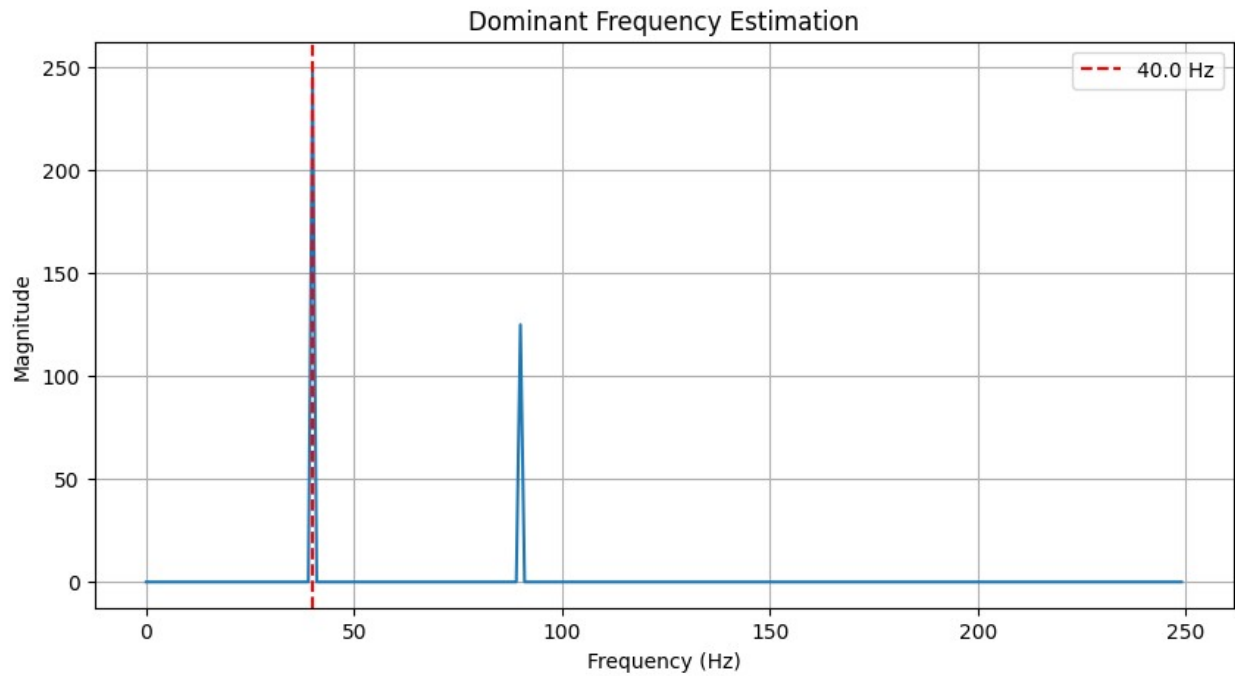
Problem 15 Hint

Use the FFT to identify the frequency corresponding to the largest spectral peak.

```
import numpy as np
import matplotlib.pyplot as plt
fs = 500
t = np.arange(0,1,1/fs)
x = np.sin(2*np.pi*40*t)+0.5*np.sin(2*np.pi*90*t)
X = np.fft.fft(x)
freq = np.fft.fftfreq(len(X),1/fs)
positive = freq >= 0
freq_pos = freq[positive]
X_pos = np.abs(X[positive])
dominant = freq_pos[np.argmax(X_pos)]
print("Estimated Dominant Frequency =",dominant,"Hz")
plt.figure(figsize=(10,5))
plt.plot(freq_pos,X_pos)
plt.axvline(dominant,color='red',linestyle='--',label=f"{dominant:.1f}")
```

```
Hz")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.title("Dominant Frequency Estimation")
plt.legend()
plt.grid(True)
plt.show()
```

Estimated Dominant Frequency = 40.0 Hz



Challenge Problems

Try these problems on your own.

Challenge Problem 2 Hint

Use a signal whose frequency is higher than half the sampling frequency.

Challenge Problem 4 Hint

Compare: Linear and Cubic interpolation

Challenge Problem 5 Hint

Round sampled amplitudes to a limited number of levels.

Challenge Problem 6

Display: Original signal, Sampled signal, FFT magnitude spectrum

```
import numpy as np
import matplotlib.pyplot as plt
f = 10
fs = 100
t = np.linspace(0,1,1000)
x = np.sin(2*np.pi*f*t)
ts = np.arange(0,1,1/fs)
xs = np.sin(2*np.pi*f*ts)
X = np.fft.fft(xs)
freq = np.fft.fftfreq(len(X),1/fs)

plt.figure(figsize=(12,10))

plt.subplot(3,1,1)
plt.plot(t,x)
plt.title("Original Signal")
plt.grid(True)

plt.subplot(3,1,2)
plt.stem(ts,xs)
plt.title("Sampled Signal")
plt.grid(True)

plt.subplot(3,1,3)
plt.plot(freq[:len(freq)//2],np.abs(X[:len(X)//2]))
plt.title("FFT Magnitude Spectrum")
plt.xlabel("Frequency (Hz)")
plt.grid(True)
```

```
plt.tight_layout()  
plt.show()
```

